Introduction to



References:
Digital Skills, Open.ED University of Edinburgh
W3schools / Python

Now let me ask something..

- Why do you want to learn Python/programming?
- What would you use Python for?

Failure

- Coding is all about trial and error.
- Don't be afraid of it.
- Error messages aren't scary, they are useful.

History

- Started by Guido Van Rossum as a hobby
- Now widely spread
- Open Source! Free!
- Versatile

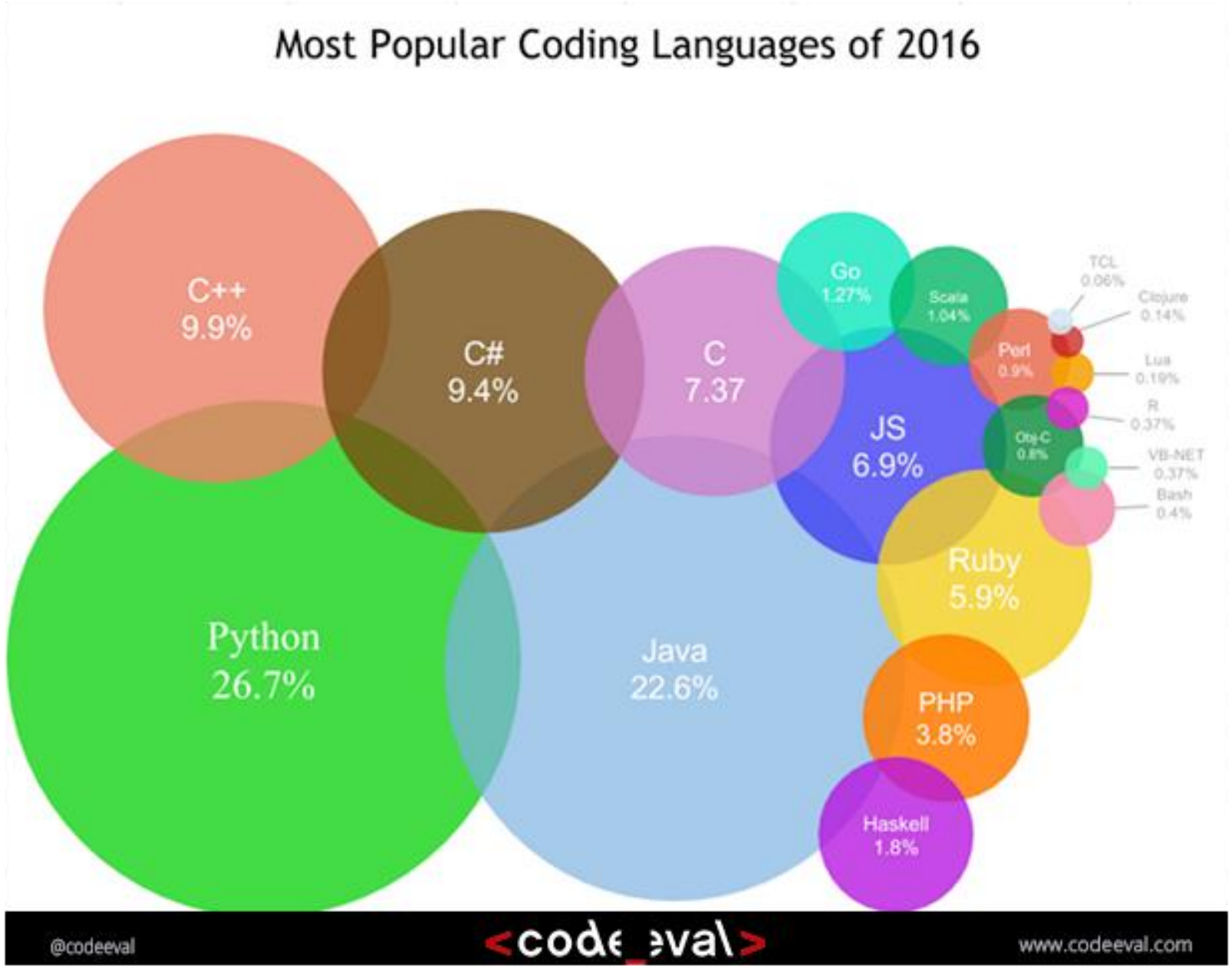


Guido Van Rossum by Doc Searls on Flickr CC-BY-SA

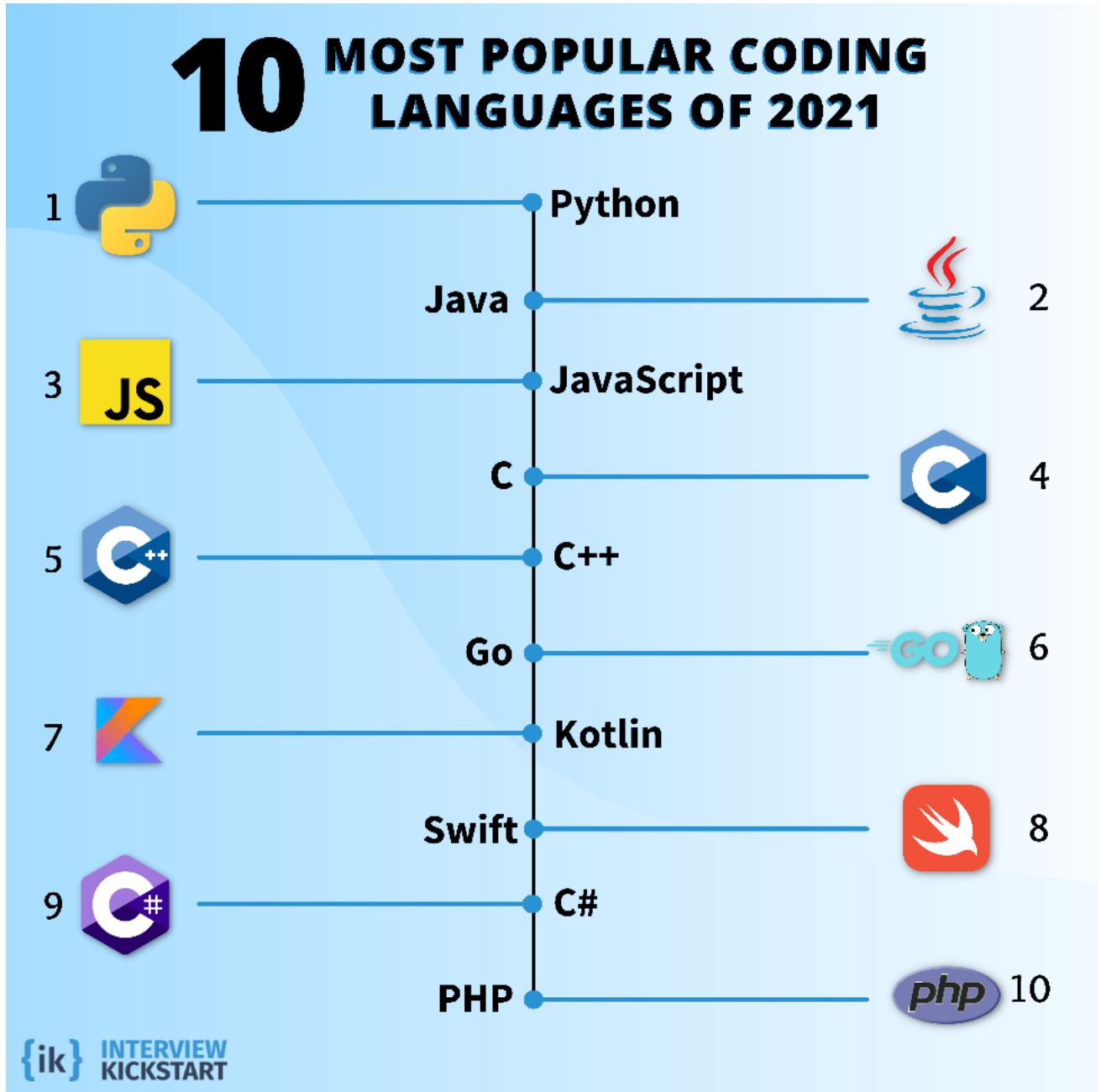
Timeline

- Python born, name picked - Dec 1989
  - By Guido van Rossum,
  - Guido is now at ***GOOGLE!!***
- First public release (USENET) - Feb 1991
- python.org website - 1996 or 1997
- 2.0 released - 2000
- Python Software Foundation - 2001
- …
- 2.4 released - 2004
- 2.5 released – 2006
- 3.9 (October 2020)
- 3.11 (2023)




Most Popular Coding Languages of 2016
C++
9.9%
C#
9.4%
C
7.37
Go
1.27%
Scala
1.04%
TCL
0.06%
Clojure
0.14%
Perl
0.9%
Lua
0.19%
R
0.37%
Obj-C
0.8%
VB.NET
0.37%
Bash
0.4%
JS
6.9%
Ruby
5.9%
Python
26.7%
Java
22.6%
PHP
3.8%
Haskell
1.8%
@codeeval
<code_eval>
www.codeeval.com

10 MOST POPULAR CODING LANGUAGES OF 2021



Python today

- Developed a large and active scientific computing and data analysis community
- Now one of the most important languages for
  - Data science
  - Machine learning
  - General software development
- Packages: NumPy, pandas, matplotlib, SciPy, scikit-learn, statsmodels

2 Modes

1. **IPython**

Python can be run interactively

Used extensively in research

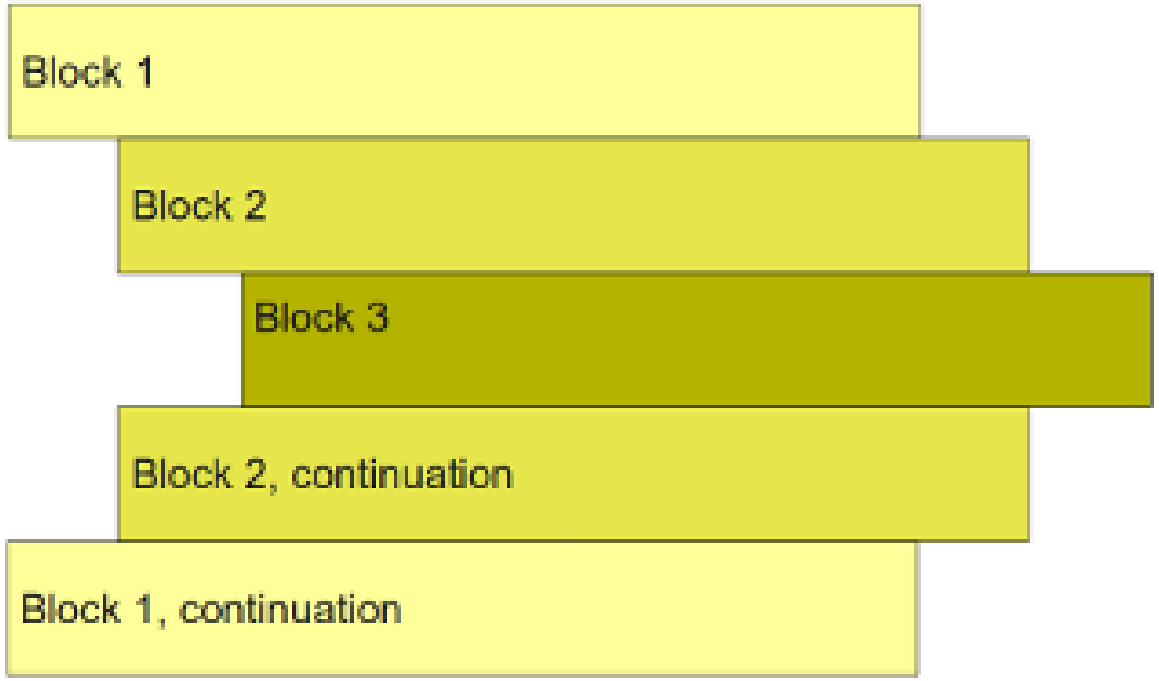
2. **Python scripts**

What if we want to run more than a few lines of code?

Then we must write text files in .py

Indentation for block structure

- How Python defines blocks?



Time for a demo..

https://www.youtube.com/watch?v=BBwEF6WBUQs

Agenda

- Comments
- Variables
- Types
- Arithmetic operators
- Boolean logic
- Strings
- Printing
- Exercises

Comments

```
#This is a comment
print("Hello, World!")
```

Comments can be placed at the end of a line, and Python will ignore the rest of the line:

```
print("Hello, World!") #This is a comment
```

Comments

A comment does not have to be text that explains the code, it can also be used to prevent Python from executing code:

```
#print("Hello, World!")
print("Cheers, Mate!")
```

Comments

Multiline Comments

```
#This is a comment
#written in
#more than just one line
print("Hello, World!")


"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

Python as a calculator

- Let us calculate the distance between Edinburgh and London in km

```
403 * 1.60934
```

```
648.56402
```

Variables

- Great calculator but how can we make it store values?
- Do this by defining variables
- Can later be called by the variable name
- Variable names are case sensitive and unique

```
distanceToLondonMiles = 403
mileToKm = 1.60934
distanceToLondonKm = distanceToLondonMiles * mileToKm
distanceToLondonKm
```

```
648.56402
```

We can now reuse the variable mileToKm in the next block without having to define it again!

```
marathonDistanceMiles = 26.219
marathonDistanceKm = marathonDistanceMiles * mileToKm
print(marathonDistanceKm)
```

```
42.19528546
```

Types

Variables actually have a type, which defines the way it is stored.

The basic types are:

Type	Declaration	Example	Usage
Integer	int	`x = 124`	Numbers without decimal point
Float	float	`x = 124.56`	Numbers with decimcal point
String	str	`x = "Hello world"`	Used for text
Boolean	bool	`x = True` or `x = False`	Used for conditional statements
NoneType	None	`x = None`	Whenever you want an empty variable

Why should we care?

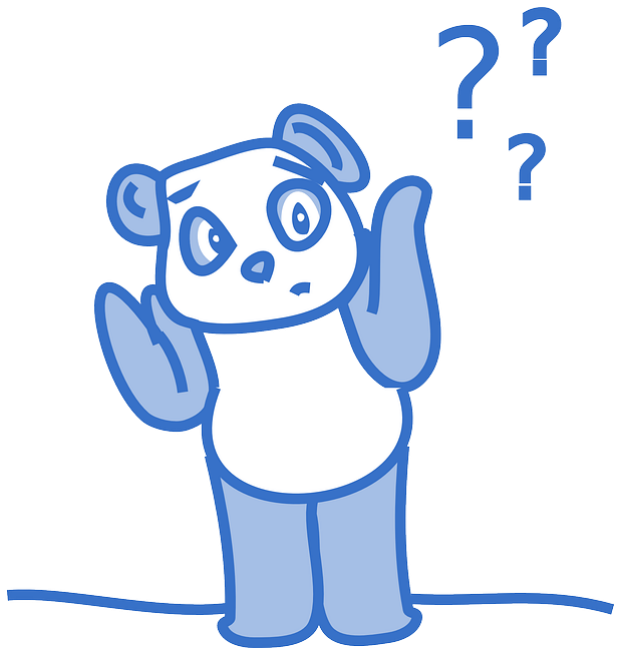


Image by Clker-Free-Vector-Images on Pixabay

```
In [4]: x = 10    # This is an integer
        y = "20"  # This is a string
        x + y
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-4-f1463b8b4c2e> in <module>()
      1 x = 10    # This is an integer
      2 y = "20"  # This is a string
----> 3 x + y

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Important lesson to remember!
We can't do arithmetic operations on variables of different types. Therefore make sure that you are always aware of your variables types!

You can find the type of a variable using **type()**. For example type **type(x)**.

Casting types

Luckily Python offers us a way of converting variables to different types!

Casting – the operation of converting a variable to a different type

```
x = 10    # This is an integer
y = "20"  # This is a string
x + int(y)
```

```
30
```

Similar methods exist for other data types: **int()**, **float()**, **str()**

Quick quiz

```
x = "10"
y = "20"
x + y
```

What will be the result?

Quick quiz

What will be the result?

```
x = 10
y = 15
x = 10.9
z = x + y
z
```

Arithmetic operations

Similar to actual Mathematics.

Order of precedence is the same as in Mathematics.

We can also use parenthesis ()

Symbol	Task Performed	Example	Result
+	Addition	4 + 3	7
-	Subtraction	4 - 3	1
/	Division	7 / 2	3.5
%	Mod	7 % 2	1
*	Multiplication	4 * 3	12
//	Floor division	7 // 2	3
**	Power of	7 ** 2	49

Order precedence example

```
16 ** 2 / 4
```

```
64.0
```

Quick quiz

```
4 + 3 ** 2
```

vs

```
(4 + 3) ** 2
```

A quick note on the increment operator shorthand

- Python has a common idiom that is not necessary, but which is used frequently and is therefore worth noting:

```
x += 1
```

Is the same as:

```
x = x + 1
```

- This also works for other operators:

```
x += y          # adds y to the value of x
x *= y          # multiplies x by the value y
x -= y          # subtracts y from x
x /= y          # divides x by y
```

Note: x++ #does not work in Python

Quiz

```
x = 10
x +=5
y = x -1
print (x)
print (y)
```

Quiz

```
x = 5
y = 10
z = 9
a = y % x
b = z % x
c = z % y
print (a)
print (b)
print (c)
```

Comparison operators

- I.e. comparison operators
- Return Boolean values

(i.e. True or False)

- Used extensively for conditional statements

Operator	Output
x == y	True if x and y have the same value
x != y	True if x and y don't have the same value
x < y	True if x is less than y
x > y	True if x is more than y
x <= y	True if x is less than or equal to y
x >= y	True if x is more than or equal to y

Comparison examples

```
x = 5        # assign 5 to the variable x
x == 5       # check if value of x is 5
```

```
True
```

Note that `==` is not the same as `=`

```
x > 7
```

```
False
```

Logical operators

- Allows us to extend the conditional logic
- Will become essential later on

Operation	Result
x or y	True if at least on is True
x and y	True only if both are True
not x	True only if x is False

a	not a
False	True
True	False

a	b	a and b	a or b
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

Truth-table definitions of bool operations

Combining both

```
x = 14
# check if x is within the range 10..20
```

True and True

```
True
```

```
x = 14
y = 42

xDivisible = ( x % 2 ) == 0 # check if x is a multiple of 2
yDivisible = ( y % 3 ) == 0 # check if y is a multiple of 3

not (xDivisible and yDivisible)
```

```
False
```

Strings

- Powerful and flexible in Python
- Can be added
- Can be multiplied
- Can be multiple lines

Strings

```
x = "Python"
y = "rocks"
x + " " + y
```

```
'Python rocks'
```

```
x = "This can be"
y = "repeated "
x + " " + y * 3
```

```
'This can be repeated repeated repeated '
```

Quiz

```
x = 5
y = 10
z = x * y
print ("the value of z is ", z)
```

Quiz

```
x = 5
y = 10
z = x * y
print ("the value of z is \n", z)
```

Some methods in string

- upper()
- lower()
- capitalize()
- count(s)
- find(s)
- rfind(s)

x = “university of SHARJAH“

Apply the methods above

Some methods in string

```
x = "university of SHARJAH"
y = x.capitalize()
z = x.lower()
a = x.upper()
b = x.count("i")
c = x.find("i") # index of "i" searching from left to right
d = x.rfind("i") # index of "i" searching from right to left. Finding the first "i"
print (y)
print (z)
print (a)
print (b)
print (c)
print (d)
```

Strings

```
x = "Edinburgh"
x = x.upper()

y = "University Of "
y = y.lower()

y + x
```

```
'university of EDINBURGH'
```

Printing

- When writing scripts, your outcomes aren't printed on the terminal.
- Thus, you must print them yourself with the print() function.
- Beware to not mix up the different type of variables!

```
print("Python is powerful!")
```

```
Python is powerful!
```

```
x = "Python is powerful"
y = " and versatile!"
print(x + y)
```

```
Python is powerful and versatile!
```

Quick quiz

Do you see anything wrong with this block?

```
str1 = "which means it has even more than"
str2 = 76
str3 = "quirks"
print(str1 + str2 + str3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call l
ast)
<ipython-input-2-3be15a6244a4> in <module>()
      2 str2 = 76
      3 str3 = " quirks"
----> 4 print(str1 + str2 + str3)

TypeError: must be str, not int
```

Another more generic way to fix it

```
str1 = "It has"
str2 = 76
str3 = "methods!"
print(str1, str2, str3)
```

```
It has 76 methods!
```

If we comma separate statements in a print function we can have different variables printing!

input() function

Example

Ask for the user's name and print it:

```
print('Enter your name:')
x = input()
print('Hello, ' + x)
```

Exercise

- Use the input() method to ask the user about the name of his/her country
- Save the name of the country in a string variable
- Count the number of ‘a’ characters
- Find the index of the letter ‘i'